SQL Aggregate Functions and Operators

(01) The students Table

1. List all distinct departments in the students table.

```
SELECT DISTINCT department
FROM students;
```

department
IT
HR
Finance

(02) Get the average age of students per department.

```
SELECT department,
       AVG(age), AS average_age
FROM students
GROUP BY department;
```

department	avg_age
IT	20,5
HR	22
Finance	23

(03) Show departments with more than 1 student

```
SELECT department,
       COUNT(*) AS student_count
FROM students
GROUP BY department
HAVING COUNT(*) > 1;
```

department	student_count
IT	2
HR	2

④ Get all students whose age is between 21 and 23

```
SELECT ~~SUM COUNT(age)~~ age,
       COUNT(*) AS student_count
FROM students
GROUP BY age
HAVING COUNT (*)
```

```
SELECT student-id, name, age
FROM students
WHERE age BETWEEN 21 AND 23;
```

student-id	name	age	department
3	Charlie	21	IT
2	Bob	22	HR
5	Eve	22	HR
4	Diana	23	Finance

⑤ List all students in the IT or HR department who are older than 21.

```
SELECT student_id, name, age, department
FROM students
WHERE department IN ('IT', 'HR')
AND age > 21;
```

Student_id	Name	age	department
2	Bob	22	HR
5	Eve	22	HR
4	Diana	23	Finance

(02) The courses Table

(06) Show total credits per department, only for departments with more than 5 total credits.

```
SELECT department,
       SUM(credits) AS total_credits
FROM courses
GROUP BY department, HAVING SUM(credits) > 5;
```

department	total_credits
IT	11

(07) List all courses that do not have 4 credits.

```
SELECT course_id, course_name, department, credits
FROM courses
WHERE credits <> 4;
```

Course_id	course_name	department	credits
101	SQL Basics	IT	3
105	Statistics	HR	3
104	Excel	Finance	2

(08) Show the top 3 courses by credits in descending order.

```
SELECT course_id, course_name, credits
FROM Courses
ORDER BY Credits DESC
LIMIT 3;
```

course_id	course_name	credits
102	Python	4
103	Data Science	4
101	SQL Basics	3

(09) The enrollments Table

(09) Get the maximum, minimum, and average grade across all enrollments.

```
SELECT MAX(grade) AS max_grade,
       MIN(grade) AS min_grade,
       AVG(grade) AS avg_grade
FROM enrollments;
```

max_grade	min_grade	avg_grade
90	78	84.6

(10) Count how many enrollments exist per course.

```
SELECT course_id, COUNT(*) AS enrollment_count
FROM enrollments
GROUP BY course_id;
```

course_id	enrollment_count
101	1
102	1
103	1
104	1
105	1

(04) The salaries Table

(11) Find total salary and total bonus per department

```
SELECT department,
       SUM(salary) AS total_salary,
       SUM(bonus) AS total_bonus
FROM   salaries
GROUP BY department;
```

department	total_salary	total_bonus
IT	122000	105000
HR	101000	7500
Finance	70000	6000

(12) Show departments where average salary is above 55000.

```
SELECT department,
       AVG(salary) AS avg_salary
FROM salaries
GROUP BY department
HAVING AVG(salary) > 55000;
```

department	avg_salary
IT	61000
HR	54500
FINance	70000

(13) List employees whose salary plus bonus is greater than 60000.

```
SELECT employee_id, name, salary, bonus, total_compensation.
FROM Salaries
WHERE (salary + bonus) > 60000;
```

employee_id	name,	salary	bonus	total_compensation
1	Tom	60000	5000	65000
3	Spike	70000	6000	76000
4	Tyke	62000	5500	67500

(05) The projects Table

(14) Show total and average budget per department. Only include department with average budgets above 70000.

```
SELECT department,
        SUM(budget) AS total_budget,
        AVG(budget) AS avg_budget
FROM projects
GROUP BY department
HAVING AVG(budget) > 70000;
```

department	total_budget	avg_budget
IT	270000	135000
Finance	80000	40000

(15) List all projects with budgets between 50000 and 120000, excluding the Marketing department.

```
SELECT project_id, project_name, department, budget
FROM project
WHERE budget BETWEEN 50000 and 120000
AND department NOT 'Marketing';
```

project_id	project_name	department	budget
1	AI App	IT	120000
2	Payroll System	Finance	80000
~~3~~			
5	HR Portal	HR	50000